

Assignment 5: Data Visualization

Sarah Hall

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
 2. Change “Student Name” on line 3 (above) with your name.
 3. Work through the steps, **creating code and output** that fulfill each instruction.
 4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
 5. Be sure to **answer the questions** in this assignment document.
 6. When you have completed the assignment, **Knit** the text and code into a single PDF file.
-

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

```
#1
#import libraries
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
library(cowplot)
```

```
##  
## Attaching package: 'cowplot'  
##  
## The following object is masked from 'package:lubridate':  
##  
##     stamp
```

```
library(lubridate)  
library(ggthemes)
```

```
##  
## Attaching package: 'ggthemes'  
##  
## The following object is masked from 'package:cowplot':  
##  
##     theme_map
```

```
#check home directory  
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
#read in processed data  
PeterPaul.chem.nutrients <-  
  read.csv(here("./Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"),  
           stringsAsFactors = TRUE)  
NIWO.litter <-  
  read.csv(here("./Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"),  
           stringsAsFactors = TRUE)  
  
#2  
#fix dates  
PeterPaul.chem.nutrients$sampldate <-  
  ymd(PeterPaul.chem.nutrients$sampldate)  
NIWO.litter$collectDate <-  
  ymd(NIWO.litter$collectDate)
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title

- Axis labels
- Axis ticks/gridlines
- Legend

```
#3
#import color ramps
library(viridis)

## Loading required package: viridisLite

library(RColorBrewer)
library(colormap)

#create my custom theme, modifying theme_minimal, make axis black, plot title blue
#legend title bold
my_theme <- theme_minimal() +
  theme(
    axis.text = element_text(
      color = "black"
    ),
    axis.line = element_line(
      color = "black",
      linewidth = 0.5
    ),
    axis.ticks = element_line(
      color = "black",
      linewidth = 0.5
    ),
    plot.title = element_text(
      color = "darkblue", size = 14, face = "bold"
    ),
    legend.title = element_text(
      color = "black", face = 'bold'
    )
  )

#set my_theme to default
theme_set(my_theme)
```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (`tp_ug`) by phosphate (`po4`), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```
#4
PeterPaul.chem.nutrients %>%
  ggplot(aes(
```

```

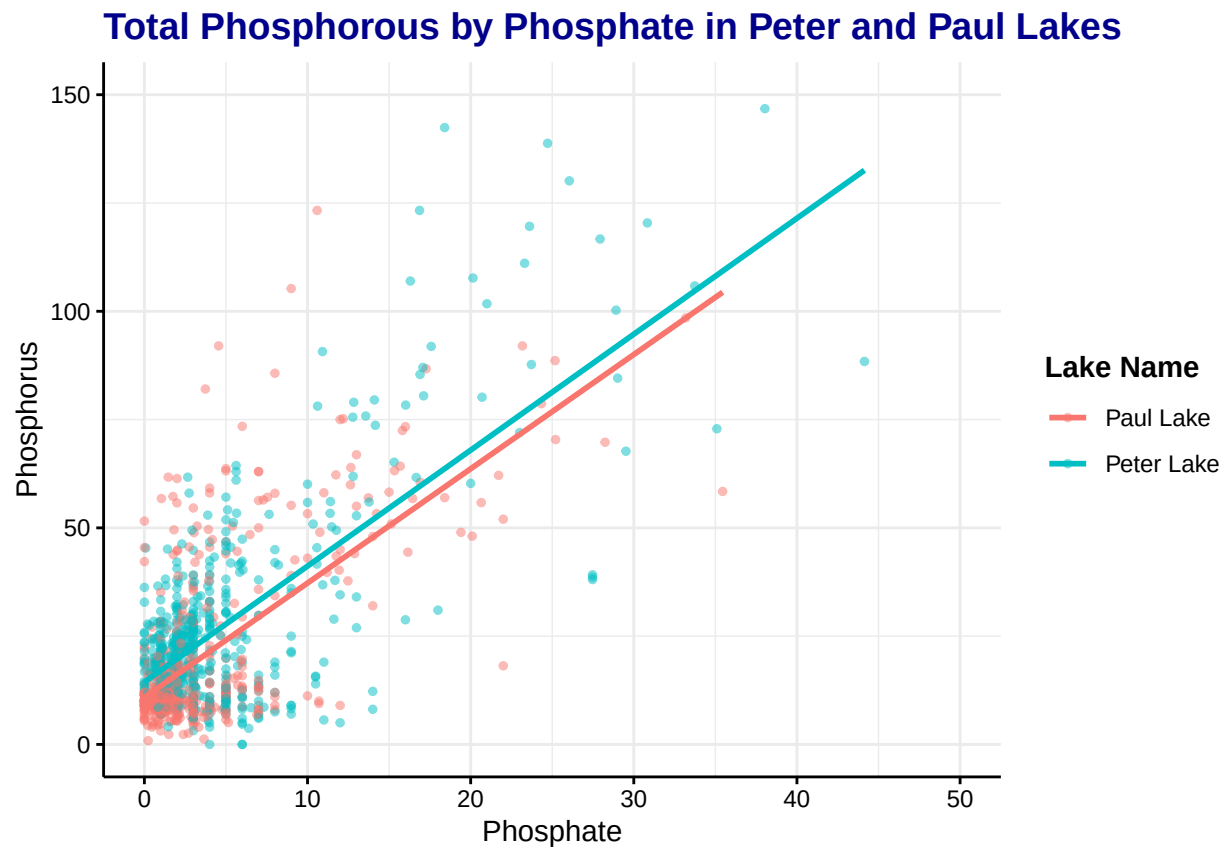
x = po4,
y = tp_ug,
color = lakename
),
) +
geom_point(alpha = 0.5, size = 1) +
xlim(0, 50) +
ylim(0, 150) +
geom_smooth(
  method = lm,
  se = FALSE
) +
labs(
  title = "Total Phosphorous by Phosphate in Peter and Paul Lakes",
  x = 'Phosphate',
  y = 'Phosphorus',
  color = 'Lake Name'
)

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 21948 rows containing non-finite outside the scale range
## ('stat_smooth()').
```

```
## Warning: Removed 21948 rows containing missing values or values outside the scale range
## ('geom_point()').
```



5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5
#set theme for the boxplots to cowplot, reduce axis text size and remove legend
theme_set(theme_cowplot() + theme (
  axis.title = element_text(size = 10),
  axis.text = element_text(size = 10),
  legend.position = "none"
))

#create temperature plot
temp_plot <- PeterPaul.chem.nutrients %>%
  ggplot(
    aes(
      x = factor(
        month,
        levels = 1:12),
      y = temperature_C,
      color = lakename
    )
  )+
  geom_boxplot()+
  scale_x_discrete(
    drop=FALSE
  )+
  labs(
    y = "Temperature (°C)",
    color = "Lake"
  )+
  theme(
    axis.title.x = element_blank(),
    axis.text.x = element_blank(),
    legend.position = ("top"),
    legend.direction = "vertical"
  )

#TP plot
TP_plot <- PeterPaul.chem.nutrients %>%
  ggplot(
    aes(
      x = factor(
        month,
        levels = 1:12),
      y = tp_ug,
      color = lakename
    )
  )+
```

```

geom_boxplot()+
scale_x_discrete(
  name = "Month",
  drop=FALSE
)+
labs(
  y = "Phosphorous (µg/L)",
)+
theme(
  axis.title.x = element_blank(),
  axis.text.x = element_blank(),
  legend.position = "none"
)

#TN plot
tn_plot <- PeterPaul.chem.nutrients %>%
ggplot(
  aes(
    x = factor(
      month,
      levels = 1:12,
      labels = month.abb),
    y = tn_ug,
    color = lakename
  )
)+
geom_boxplot()+
scale_x_discrete(
  name = "Month",
  drop=FALSE
)+
labs(
  y = "Nitrogen (µg/L)",
)+
theme(
  legend.position = "none"
)

#combine to one frame
final_plot <-
plot_grid(
  temp_plot, TP_plot, tn_plot,
  ncol = 1,
  align = "v" #stack vertically
)

```

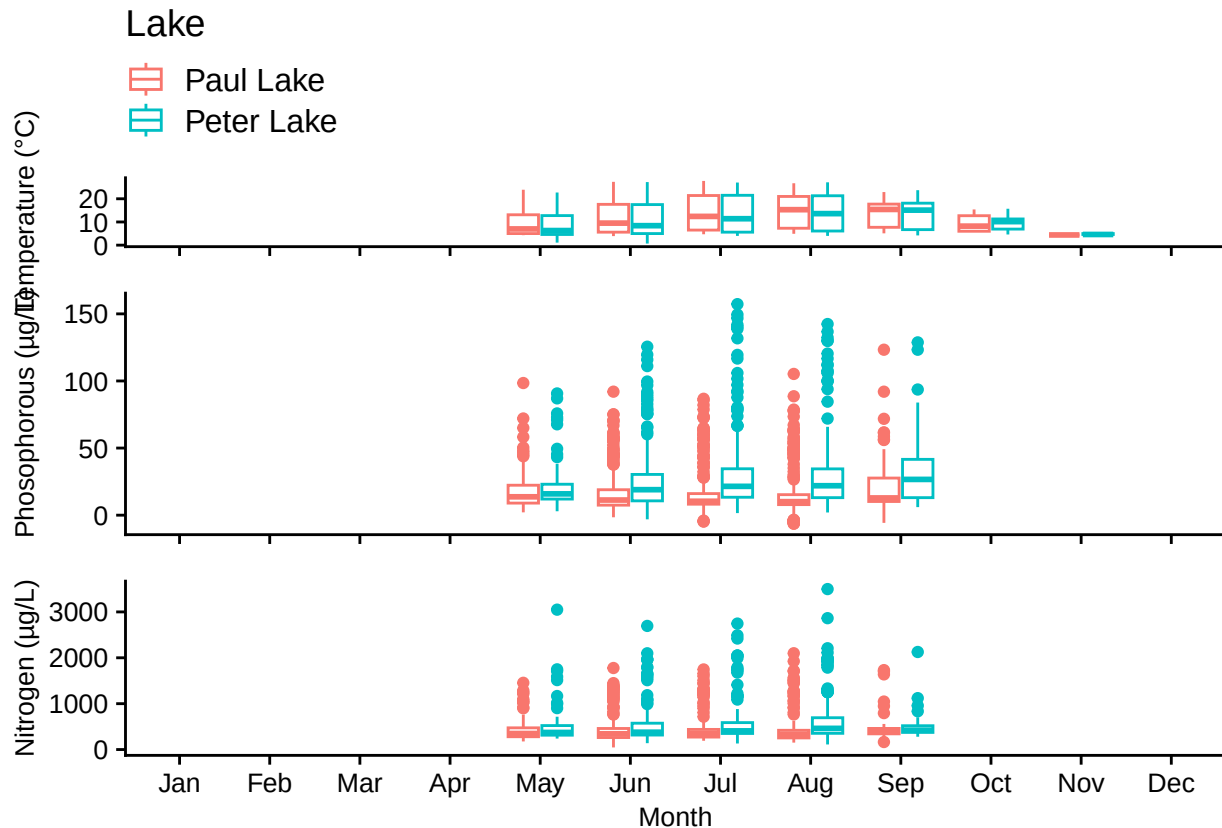
```
## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
```

```
## ('stat_boxplot()').
```

```
#print plot
final_plot
```



Question: What do you observe about the variables of interest over seasons and between lakes?

Answer: First, the data was only collected between May and September. The lakes are the warmest in the mid-summer, peaking in July and August. The two lakes have very similar temperature patterns. The total phosphorous appears to increase throughout the summer, peaking in September. Peter Lake has higher phosphorous concentrations than Paul Lake. Peter Lake also has higher nitrogen concentrations than Paul Lake. It's a little difficult to discern a trend in nitrogen, but they appear to be increase in the summer as well, peaking in August.

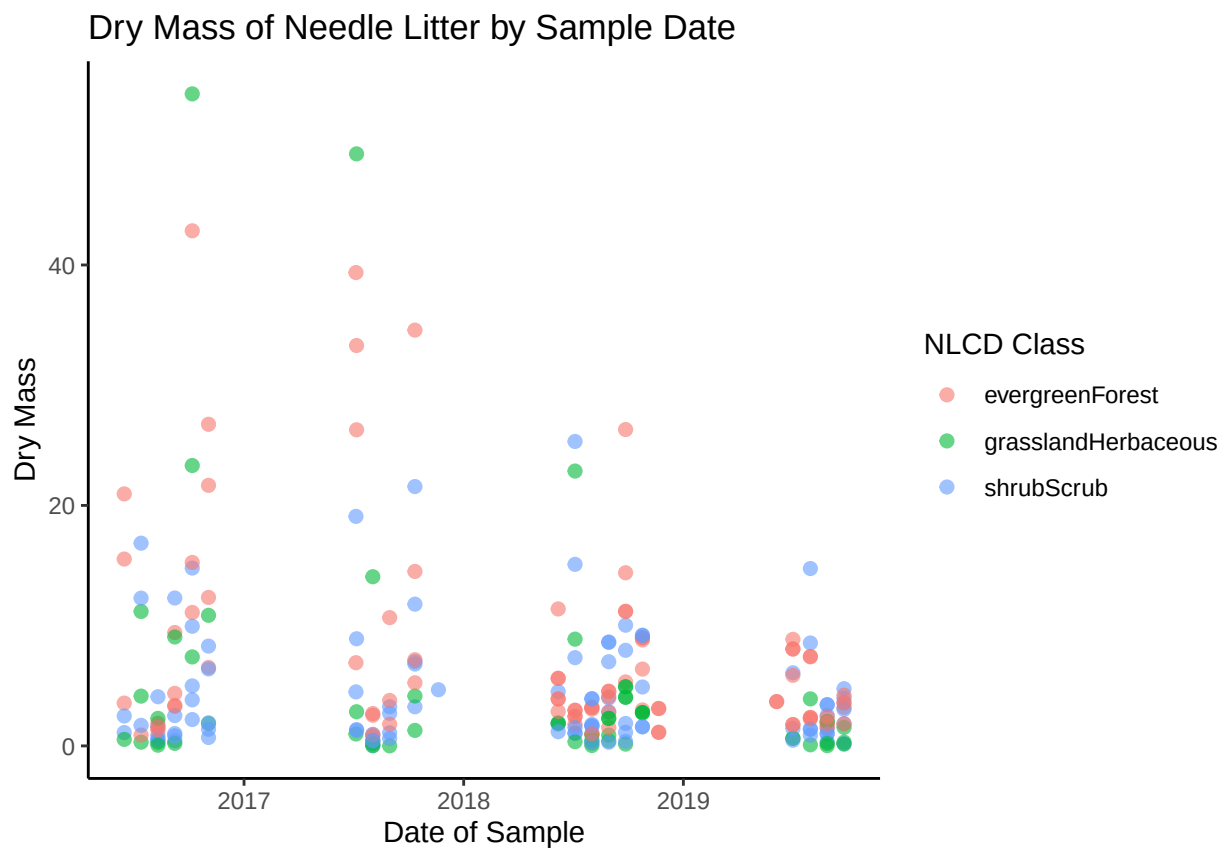
6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

```
#6
NIWO.litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot()
```

```

aes(
  x = collectDate,
  y = dryMass,
  color = nlcdClass
)
)+
geom_point(alpha = 0.6, size = 2)+
labs(
  title = "Dry Mass of Needle Litter by Sample Date",
  y = "Dry Mass",
  x = "Date of Sample",
  color = "NLCD Class"
) +
theme_classic() #use classic theme

```



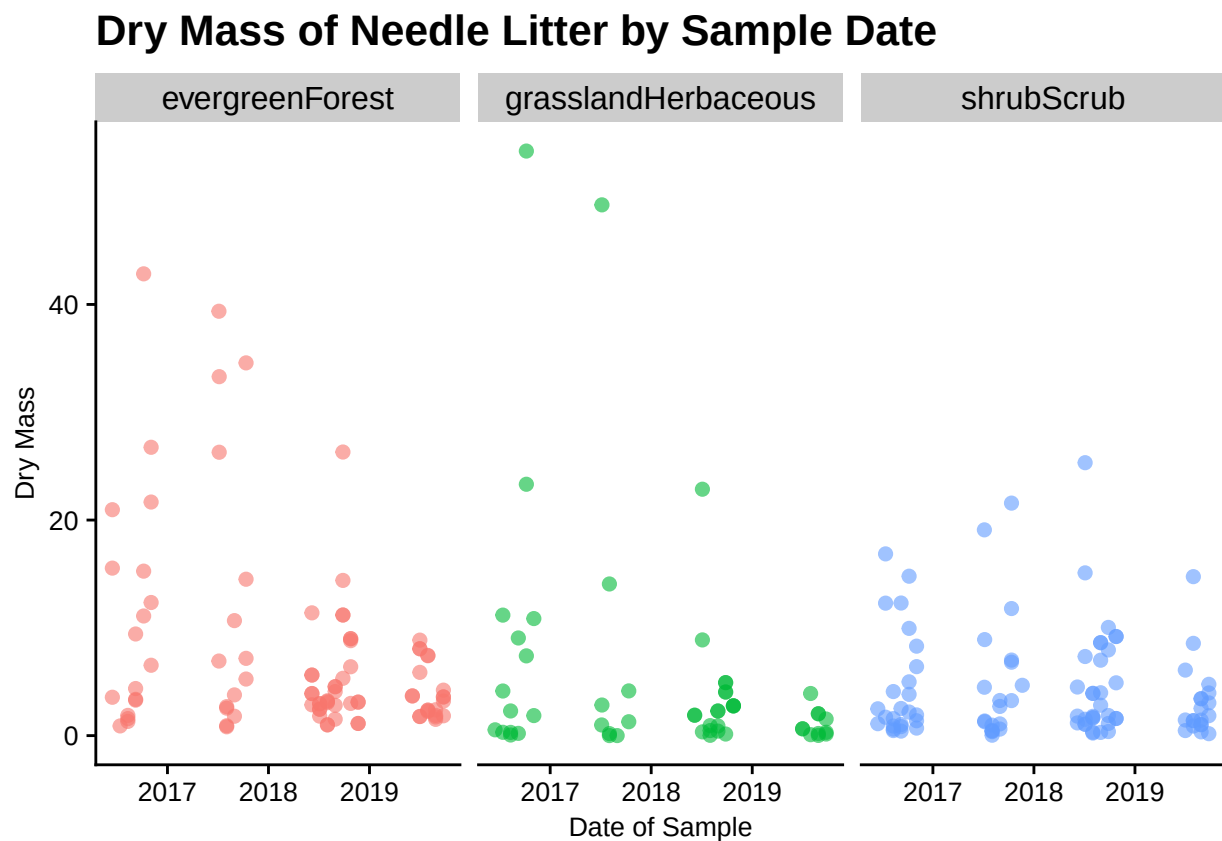
```

#7
NIWO.litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot(
    aes(
      x = collectDate,
      y = dryMass,
      color = nlcdClass
    )
  ) +

```



```
geom_point(alpha = 0.6, size = 2) +
facet_wrap(
  facets = vars(nlcdClass),
  nrow = 1,
) +
labs(
  title = "Dry Mass of Needle Litter by Sample Date",
  y = "Dry Mass",
  x = "Date of Sample"
)
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: 7 is more effective because it avoids the overplotting that occurs with 6. It helps spread out the values so you can clearly see where there are clusters for each specific class, as well as easily compare across the three classes with the shared y-axis.